

Search Engine Project Report

News Category Dataset
Information Retrieval & Data Mining

Course: Data Mining

Implementation: Python (Jupyter Notebook)

*This report describes each stage of the search engine implementation,
from data collection through evaluation.*

Table of Contents

1. **Introduction**
2. **Phase 1: Data Collection, Preprocessing & Indexing**
 - 2.1 Data Collection (Step 1)
 - 2.2 Text Preprocessing (Step 2)
 - 2.3 Inverted Index Construction (Step 3)
3. **Phase 2: Query Processing, Expansion, UI & Evaluation**
 - 3.1 Query Processing with TF-IDF Ranking (Step 4)
 - 3.2 Query Expansion (Step 5)
 - 5a. Pseudo-Relevance Feedback (PRF)
 - 5b. WordNet Synonym Expansion
 - 5c. BERT-Based Semantic Expansion
 - 3.3 User Interface (Step 6)
 - 3.4 Evaluation (Step 7)
4. **Conclusion**

1. Introduction

This report documents the design and implementation of a basic search engine built for a large collection of news articles. The system follows a two-phase development approach as specified in the course project requirements:

- Phase 1 covers data collection, text preprocessing, and inverted index construction.
- Phase 2 covers query processing with TF-IDF ranking, query expansion using multiple techniques (Pseudo-Relevance Feedback, WordNet synonyms, and BERT embeddings), a user interface, and performance evaluation.

The entire implementation is contained in a single Jupyter Notebook (Final.ipynb) written in Python 3. Key libraries used include NLTK for natural language processing, Sentence-Transformers for BERT-based embeddings, scikit-learn for cosine similarity computation, and ipywidgets for the interactive user interface.

2. Phase 1: Data Collection, Preprocessing & Indexing

2.1 Data Collection (Step 1)

Dataset

The corpus is the "News Category Dataset v3" (News_Category_Dataset_v3.json), a publicly available dataset sourced from online repositories. It contains 209,527 news articles from HuffPost spanning multiple categories.

Format

The dataset is stored as a JSON-lines file where each line is a JSON object. Each record contains the following fields:

- link - URL to the original article
- headline - article headline
- category - news category label (e.g., POLITICS, ENTERTAINMENT)
- short_description - brief summary of the article
- authors - article author(s)
- date - publication date

Corpus Construction

The dataset is loaded line-by-line, parsing each JSON record. For each document, the headline and short_description fields are combined into a single text field to form the searchable content. Each document is assigned a unique sequential ID (0 to 209,526). The original record is preserved for display purposes.

Key code snippet for data loading:

```
combined_text = f"{headline} {short_description}"
data.append({
    'id': doc_id,
    'text': combined_text,
    'original_record': record
})
```

Total documents loaded: 209,527

2.2 Text Preprocessing (Step 2)

All documents pass through a five-stage preprocessing pipeline implemented in the preprocess_text() function. The same pipeline is applied to both documents (during indexing) and queries (during retrieval) to ensure consistency.

Pipeline Steps

- Lowercasing: All text is converted to lowercase to ensure case-insensitive matching.
- Special Character Removal: Non-alphanumeric characters are removed using regex (re.sub(r'^a-z0-9\s', "", text)). Multiple whitespaces are collapsed.
- Tokenization: Text is split into individual word tokens using NLTK's word_tokenize().
- Stopword Removal: Common English stop words are removed using NLTK's built-in stopwords list (e.g., 'the', 'is', 'and', 'of').
- Stemming: Words are reduced to their root forms using the Porter Stemmer (e.g., "running" -> "run", "Americans" -> "american").

Example transformation:

```
Original: "Over 4 Million Americans Roll Up Sleeves For..."
Preprocessed: ['4', 'million', 'american', 'roll', 'sleeve',
               'omicrontarget', 'covid', 'booster', ...]
```

2.3 Inverted Index Construction (Step 3)

An inverted index is the core data structure that maps each unique term to the list of documents containing it, along with the term frequency in each document.

Data Structure

The index is implemented as a Python dictionary where each key is a stemmed term and each value is a list of [document_id, frequency] pairs, sorted by document ID.

```
inverted_index = {
    'covid': [[0, 1], [206, 1], [335, 1], [438, 1], [470, 2], ...],
    'health': [[55, 1], [141, 1], [311, 1], ...],
    ...
}
```

Construction Process

The build_inverted_index() function iterates over all documents, preprocesses each document's text, and for every token, increments the term frequency count in a nested defaultdict. After processing all documents, the temporary structure is converted to sorted posting lists.

Index Statistics

Total unique terms: 83,462

Total documents indexed: 209,527

Top 5 most frequent terms in the corpus:

- 1. "trump" - 20,046 total occurrences
- 2. "new" - 19,672 total occurrences
- 3. "one" - 17,049 total occurrences
- 4. "us" - 15,154 total occurrences
- 5. "get" - 14,627 total occurrences

3. Phase 2: Query Processing, Expansion, UI & Evaluation

3.1 Query Processing with TF-IDF Ranking (Step 4)

Query Pipeline

User queries are processed through the same preprocessing pipeline as documents (lowercase, tokenization, stopword removal, stemming) to ensure term matching. The process_query() function implements the following retrieval logic:

- Term Matching (AND logic): For each query term, the inverted index is consulted to retrieve the set

of documents containing that term. Only documents containing ALL query terms are considered candidates.

- TF-IDF Scoring: Each candidate document is scored using the TF-IDF weighting scheme.

TF-IDF Computation

The scoring combines two components:

- Term Frequency (TF): Computed as $\text{count}(\text{term}, \text{doc}) / \text{len}(\text{doc_tokens})$, representing the normalized frequency of the term in the document.
- Inverse Document Frequency (IDF): Computed as $\log(N / \text{df})$, where N is the total number of documents and df is the number of documents containing the term.

The final score for a document is the sum of TF*IDF across all query terms. Results are sorted by descending score and the top-K are returned.

Example query results:

```
Query: 'covid vaccine health'
Doc 2307 | Score: 1.1191 | Marjorie Taylor Greene Wants Chinese...
Doc 935 | Score: 0.8139 | FDA Authorizes Second COVID Booster...
Doc 2297 | Score: 0.7162 | Florida Again Leads Nation in Soaring...
```

3.2 Query Expansion (Step 5)

Three query expansion techniques are implemented to improve retrieval by enriching the original query with related terms.

5a. Pseudo-Relevance Feedback (PRF)

Standard Pseudo-Relevance Feedback assumes that the top-K results from an initial retrieval are relevant. The system extracts discriminative terms from these documents to expand the query. The implementation follows these steps:

1. Run the initial query using the standard TF-IDF retrieval.
2. Take the top-K results (default K=20) as pseudo-relevant documents.
3. Extract non-query terms from these documents, scored by their TF-IDF weight.
4. Select the top-N expansion terms (default N=5).
5. Re-retrieve using the expanded query with OR logic to bring in new documents.

Example PRF expansion:

```
Query: 'covid health'
Expansion terms: ['booster', 'jha', 'variant', 'vaccin', 'mental']

Query: 'election president'
Expansion terms: ['elector', 'poland', 'burundi', 'oliv', 'endors']
```

5b. WordNet Synonym Expansion

The `expand_query_with_synonyms()` function uses NLTK's WordNet lexical database to find synonyms for each query term. For each token, the function retrieves up to 3 synsets and 2 lemmas per synset, stems the synonym, and adds it to the expanded query if it is not already present. A fallback manual synonym map is

provided in case WordNet is unavailable.

Example synonym expansion:

```
Query: 'covid health'
Original tokens: ['covid', 'health']
Synonyms added: ['wellness']
Expanded tokens: ['well', 'health', 'covid']
```

5c. BERT-Based Semantic Expansion

The most advanced expansion method uses sentence-level BERT embeddings to find semantically similar terms from the indexed vocabulary. The implementation uses the 'all-MiniLM-L6-v2' model from the Sentence-Transformers library.

Process

- The top 15,000 most frequent vocabulary terms are pre-computed and their BERT embeddings are stored.
- When a query is received, its BERT embedding is computed.
- Cosine similarity is calculated between the query embedding and all vocabulary term embeddings.
- The top-N most similar terms (excluding original query terms) are returned as expansion candidates.

Example BERT expansion:

```
Query: 'covid health'
covid19      (similarity: 0.6637)
flu          (similarity: 0.5410)
influenza    (similarity: 0.5409)
coronavirus  (similarity: 0.5193)

Query: 'election president'
presidentelect (similarity: 0.6343)
clinton       (similarity: 0.6336)
obama         (similarity: 0.6257)
bush          (similarity: 0.6170)
trump         (similarity: 0.6144)
```

3.3 User Interface (Step 6)

An interactive user interface is built using Jupyter's ipywidgets library, allowing users to interact with the search engine directly within the notebook.

UI Components

- Text Input: A search box where users type their query (e.g., 'covid vaccine health').
- Results Slider: An integer slider (1-20) to control the number of results returned.
- Expansion Dropdown: A dropdown menu with four options: 'None' (basic TF-IDF), 'Relevance Feedback', 'BERT Embeddings', and 'Both (RF + BERT)'.
- Search Button: Triggers the search when clicked.
- Output Area: Displays results in a styled HTML table showing rank, headline (as clickable link),

category, date, description snippet, and TF-IDF score.

Features

- Real-time query processing with response time display.
- When using BERT expansion, the expansion terms are displayed to the user.
- Error handling with traceback display for debugging.
- Clean, Google-inspired visual styling with alternating row colors.

3.4 Evaluation (Step 7)

The search engine is evaluated using standard Information Retrieval metrics across six test queries and three retrieval methods.

Evaluation Methodology

Since the dataset does not include ground-truth relevance judgments, pseudo-relevance judgments (pseudo-qrels) are constructed using category matching. A query-to-category mapping is defined (e.g., 'covid' -> WELLNESS, 'election' -> POLITICS, 'movie' -> ENTERTAINMENT), and a document is considered relevant if its category matches any of the query's mapped categories.

Metrics

- Precision@10: Fraction of retrieved documents in the top-10 that are relevant.
- Recall: Fraction of all relevant documents that were retrieved (note: denominators are large due to category-based relevance).
- F1-Score: Harmonic mean of precision and recall.
- Response Time: Wall-clock time for query processing.

Test Queries

- "covid health"
- "election president"
- "movie film"
- "stock market"
- "travel food"
- "game sport"

Evaluation Results Summary

Method	Avg P@10	Avg F1	Avg Time
Basic TF-IDF	0.4500	0.000802	0.0306s
Standard PRF	0.2500	0.000540	1.7654s
BERT Expansion	0.5000	0.000968	1.7764s

Note on F1-Score Values

The F1 scores appear extremely small (on the order of 0.001) across all methods. This is not an error but a direct consequence of how recall is computed in this evaluation setup. Since relevance is determined by

category matching, the number of relevant documents per query is very large (each news category contains thousands to tens of thousands of articles out of the ~209,000 total documents). Because the system only retrieves the top-10 results, the recall denominator dwarfs the numerator (e.g., retrieving 5 relevant documents out of 18,000 relevant documents yields a recall of ~0.0003). Since F1 is the harmonic mean of precision and recall, the extremely low recall pulls the F1 score down to near zero even when precision is reasonable. Precision@10 is therefore the most meaningful metric in this evaluation.

Key Findings

- BERT Expansion achieves the highest average Precision@10 (0.50), outperforming both Basic TF-IDF (0.45) and Standard PRF (0.25).
- Basic TF-IDF is the fastest method (0.03s average), while PRF and BERT expansion require approximately 1.77s due to additional processing.
- Standard PRF shows lower precision, likely because expansion terms sometimes introduce topic drift (e.g., expanding 'covid health' with 'mental' leads to unrelated mental health articles).
- BERT expansion produces more semantically coherent expansion terms (e.g., 'covid' -> 'covid19', 'flu', 'influenza', 'coronavirus'), leading to better precision.

4. Conclusion

This project successfully implements a complete search engine pipeline for the News Category Dataset containing 209,527 documents. The system covers all required phases:

- Phase 1: Data collection from a public JSON-lines dataset, five-stage text preprocessing (lowercasing, special character removal, tokenization, stopword removal, stemming), and construction of an inverted index with 83,462 unique terms.
- Phase 2: TF-IDF-based query processing with AND retrieval logic, three query expansion techniques (Pseudo-Relevance Feedback, WordNet synonyms, and BERT sentence embeddings), an interactive Jupyter-based user interface with configurable parameters, and systematic evaluation using Precision@10, Recall, F1, and response time.

The evaluation demonstrates that BERT-based expansion provides the best retrieval quality (Precision@10 = 0.50) by leveraging semantic similarity to find contextually relevant expansion terms. The basic TF-IDF method remains the fastest option for latency-sensitive applications, while Pseudo-Relevance Feedback offers a middle ground that does not require a pre-trained model.

Technologies Used

Language:	Python 3.12
Environment:	Jupyter Notebook
NLP:	NLTK (tokenization, stemming, stopwords, WordNet)
Embeddings:	Sentence-Transformers (all-MiniLM-L6-v2 / BERT)
Similarity:	scikit-learn (cosine_similarity)

UI: ipywidgets + IPython.display (HTML rendering)
Dataset: News Category Dataset v3 (209,527 articles)